

Caleb Schexnider
Darrin Rious
Justin Le
Roland Okungbowa

C00507861
C00520576
C00538360
C00476033

Task 1:

1. - The only chance for a deadlock would be a case where operations that need to be done cannot be performed by any domain due to random chance. To get this to occur, a reading or writing function would need to be called with no chance of reaching it due to either it not existing or not being able to get to the domain which can perform this function.

Task 2:

1. - This task differs in the way of utilizing a series of linked lists to display the data compared to Task 1 using the 2D array. It was easier to implement due to the lack of only needing to search the single list for the data that is required, rather than searching an entire 2D array.

Task 3:

1. - Capability lists are much more complex due to the fact that they are not object-based. This makes the process of revoking access more complex, and it also makes the OS have to do more work to access the pointer rather than access it directly. There's an additional workload there due to that complexity and the fact the structure requires checking every single domain's privileges every time rather than a simple check for the privileges of a single domain like in an Access list.

Task 4:

1. - The multi-user system is important for the integrity of the data. There should not be a case where multiple users are able to access and adjust the data or system simultaneously, as this could lead to conflicting edits or information changing mid-edit. If there were to be a case where users were all granted equal permissions, then there could be an issue where tasks that should be queued up are done simultaneously which can lead to data changing during an action that requires that data (e.g. writing while data is being read). Protection also controls the authorized usage of files to ensure that the system is not illegally modified. Equal permissions would allow the illegitimate access or modifications of files which would risk the overall security of the system

Task 5:

1. - Task 1 was first implemented by generating a 2d array, internal components were randomly generated determining object and domain permissions. Task 2 utilized linked lists holding numerous linked lists with objects. Task 3 also utilized the same method but instead contained domains, making the structure somewhat more difficult to implement. All 3 run methods for the tasks consisted of randomly generating a number to first consider domain switching then

considering the action within a file. Then the run method would internally check within by determining whether access was authorized or illegal. Some minor bugs occurred due to incorrect matrix iteration, later fixed to correctly display objects and domains

2. - Task 1 primarily used arrays as the main data structures, with the access matrix composed of a 2d array. Task 2 and Task 3 both used linked lists containing more linked lists to store objects and domains respectively. All 3 tasks used arrays to hold colors and locks. The algorithm for the run method consisted of determining action, checking if authorized, and looping at least 5 times